Simple Linux character device driver – Oleg Kutkov personal blog

oleg_kutkov



A character device is one of the simplest ways to communicate with a module in the Linux kernel.
These devices are presented as special files in a */dev* directory and support direct reading and writing of any data, byte by byte, like a stream.
Actually, most of the pseudo-devices in */dev* are character devices: serial ports, modems, sound, and video adapters, keyboards, some custom I/O interfaces. Userspace programs can easily open, read, write, and custom control requests with such device files.
Here I am describing how to write a simple Linux kernel module which can create one or multiple character device.

Introducing to character devices.

Detection of the device type in */dev* directory is pretty simple.

```
$ ls -l /dev/ttyS0
crw-rw---- 1 root dialout 4, 64 Mar 11 16:52 /dev/ttyS0
```

Symbol **C,** in the beginning, means that this device is a character device.
Also, you can find here two strange numbers: **4** and **64**. This is a Major and Minor number of this device.
Inside the Linux kernel, every device is identified not by a symbolic name but by a unique number – the device's major number. This number assigning by the kernel during device registration. Every device driver can support multiple "sub-devices". For example, a serial port adapter may contain two hardware ports. Both of these ports are handled by the same driver, and they share one Major number. But inside this driver, each of these ports is also identified by the unique number, and this is a device Minor number.

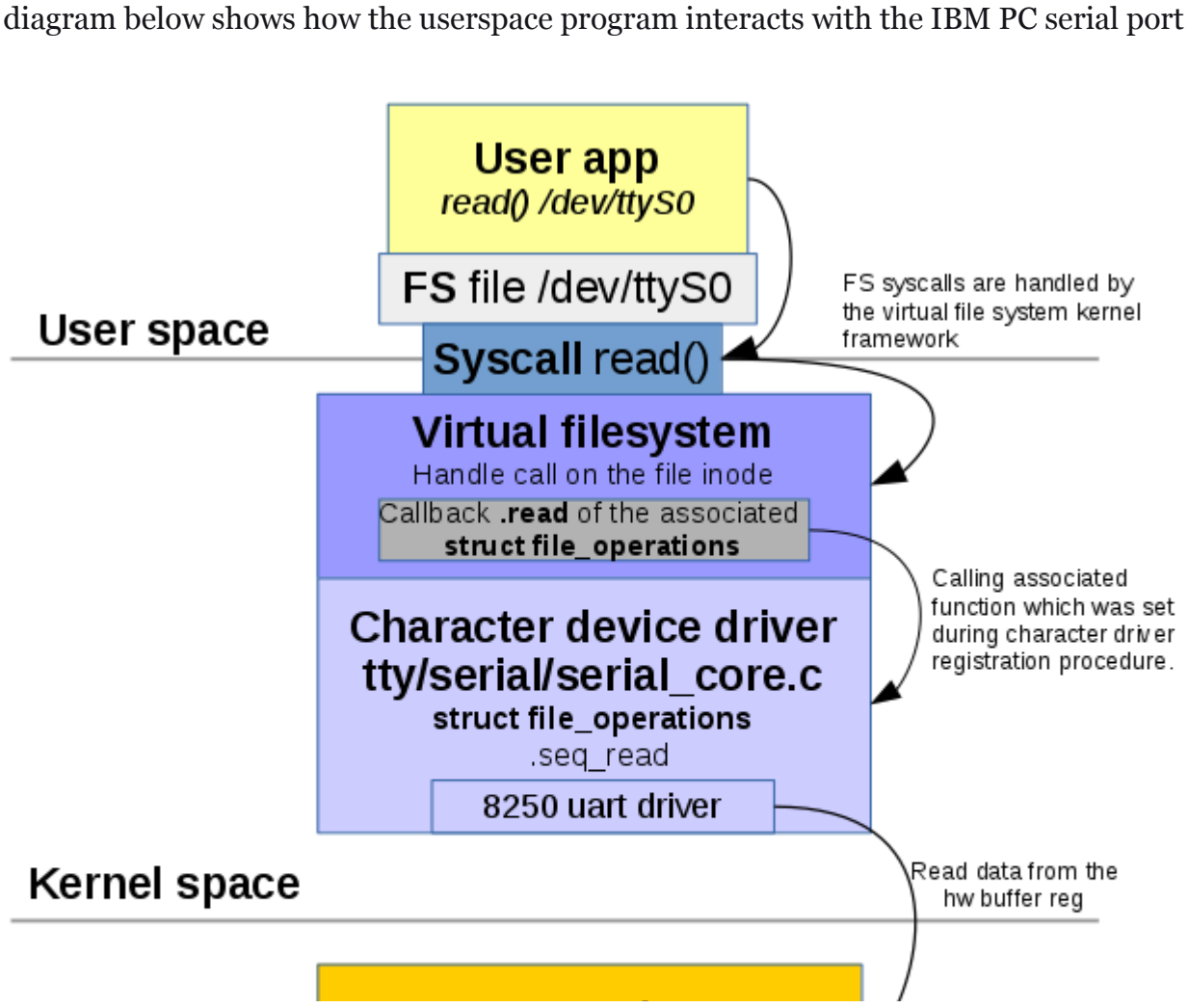
```
crw-rw---- 1 root dialout 4, 64 Mar 11 16:52 /dev/ttyS0
crw-rw---- 1 root dialout 4, 65 Mar 11 16:52 /dev/ttyS1
crw-rw---- 1 root dialout 4, 66 Mar 11 16:52 /dev/ttyS2
```

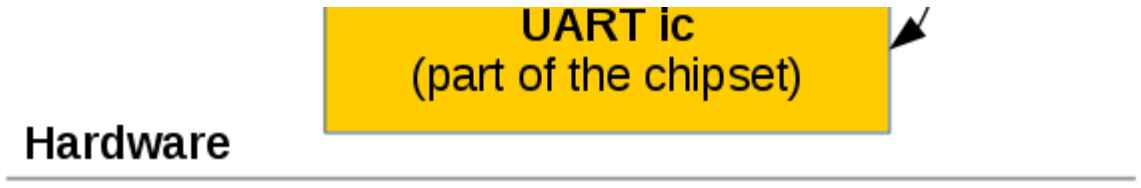
One Major number **4** for every ttySX device and different (**64**–**65**) Minor numbers.
The driver's code assigns minor numbers, and the developer of this driver may select any suitable values.

As this device acts like a file – programs can do almost everything except seeking. Every file operation on this object commands the driver to do something inside the Linux kernel and start reading some data from the hardware.

At the end of this article, you can find a complete example of the character device driver, but first, let's discuss how it works.

The diagram below shows how the userspace program interacts with the IBM PC serial port using the character device.





The virtual filesystem is an abstraction layer on top of a more concrete file system. A VFS aims to allow client applications to access different types of concrete file systems uniformly.

File operations

In special device files, VFS is responsible for calling I/O functions set by the device driver.
To set this function special kernel structure is used.

```
struct file_operations {
  struct module *owner;
  loff_t (*llseek) (struct file *, loff_t, int);
  ssize_t (*read) (struct file *, char *, size_t, loff_t *);
  ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
  int (*readdir) (struct file *, void *, filldir_t);
  unsigned int (*poll) (struct file *, struct poll_table_struct *);
  int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
  int (*mmap) (struct file *, struct vm_area_struct *);
  int (*open) (struct inode *, struct file *);
  int (*flush) (struct file *);
  int (*release) (struct inode *, struct file *);
  int (*fsync) (struct file *, struct dentry *, int datasync);
  int (*fasync) (int, struct file *, int);
  int (*lock) (struct file *, int, struct file_lock *);
   ssize_t (*readv) (struct file *, const struct iovec *, unsigned long,
    loff_t *);
   ssize_t (*writev) (struct file *, const struct iovec *, unsigned long,
    loff_t *);
};
```

Some operations are not implemented by a driver. For example, a driver that handles a video card won't need to read from a directory structure. The corresponding entries in the **file_operations** structure should be set to `NULL`.

In a C99 way, initialization is simple.

```
struct file_operations fops = {
  .read = device_read,
  .write = device_write,
  .open = device_open,
  .release = device_release
};
```

Initialized **file_operation**s can be assigned to the character device during device registration.

Registration of the character device

The registration procedure consists of several simple steps.
First, you need to decide how many minor devices you need. This is a constant which typically depends on your hardware (if you are writing a driver for real hardware).
Minor numbers are convenient to use as part of the device name. For example,/dev/mychardev0 with a Minor 0 /dev/mychardev2 with a Minor 2.

The first step is an allocation and registration of the range of char device numbers using *alloc_chrdev_region*.

int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count, char *name);

Where *dev* is output parameter for first assigned number, *baseminor* is first of the requested range of minor numbers (e.g., 0), *count* is a number of minor numbers required, and *name* – the associated device's name driver.

The major number will be chosen dynamically and returned (along with the first minor number) in dev.
The function returns zero or a negative error code.

To get generated Major number, we can use **MAJOR()** macros.

int dev_major = MAJOR(dev);

Now it's time to initialize a new character device and set file_operations with *cdev_init*.

void cdev_init(struct cdev *cdev, const struct file_operations *fops);

struct cdev represents a character device and is allocated by this function.

Now add the device to the system.

int cdev_add(struct cdev *p, dev_t dev, unsigned count);

Finally – create a device file node and register it with sysfs.

struct device * device_create(struct class *class, struct device *parent, dev_t devt, const char *fmt, ...);

Now all together.
This code creates 2 character devices with names */dev/mychardev0* and */dev/mychardev1*

```
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/kernel.h>
#include <linux/fs.h>

// max Minor devices
#define MAX_DEV 2

// initialize file_operations
static const struct file_operations mychardev_fops = {
   .owner      = THIS_MODULE,
   .open       = mychardev_open,
   .release    = mychardev_release,
   .unlocked_ioctl = mychardev_ioctl,
   .read       = mychardev_read,
   .write      = mychardev_write
};

// device data holder, this structure may be extended to hold additional data
struct mychar_device_data {
   struct cdev cdev;
};

// global storage for device Major number
static int dev_major = 0;

// sysfs class structure
static struct class *mychardev_class = NULL;

// array of mychar_device_data for
static struct mychar_device_data mychardev_data[MAX_DEV];

void mychardev_init(void)
{
   int err, i;
   dev_t dev;

   // allocate chardev region and assign Major number
   err = alloc_chrdev_region(&dev, 0, MAX_DEV, "mychardev");

   dev_major = MAJOR(dev);

   // create sysfs class
```

```
    mychardev_class = class_create(THIS_MODULE, "mychardev");

    // Create necessary number of the devices
    for (i = 0; i < MAX_DEV; i++) {
        // init new device
        cdev_init(&mychardev_data[i].cdev, &mychardev_fops);
        mychardev_data[i].cdev.owner = THIS_MODULE;

        // add device to the system where "i" is a Minor number of the new device
        cdev_add(&mychardev_data[i].cdev, MKDEV(dev_major, i), 1);

        // create device node /dev/mychardev-x where "x" is "i", equal to the Minor number
        device_create(mychardev_class, NULL, MKDEV(dev_major, i), NULL, "mychardev-%d", i);
    }
}
```

You can find a few new things in this example. The creation of the sysfs class is a necessary part of the device node creation.
Function class_create(THIS_MODULE, “mychardev”) creates sysfs class with paths for each character devices:

```
$ tree /sys/devices/virtual/mychardev/
/sys/devices/virtual/mychardev/
├── mychardev-0
│   ├── dev
│   ├── power
│   │   ├── async
│   │   ├── autosuspend_delay_ms
│   │   ├── control
│   │   ├── runtime_active_kids
│   │   ├── runtime_active_time
│   │   ├── runtime_enabled
│   │   ├── runtime_status
│   │   ├── runtime_suspended_time
│   │   └── runtime_usage
│   ├── subsystem -> ../../../../class/mychardev
│   └── uevent
└── mychardev-1
    ├── dev
    ├── power
    │   ├── async
    │   ├── autosuspend_delay_ms
    │   ├── control
    │   ├── runtime_active_kids
    │   ├── runtime_active_time
    │   ├── runtime_enabled
    │   ├── runtime_status
    │   ├── runtime_suspended_time
    │   └── runtime_usage
    ├── subsystem -> ../../../../class/mychardev
    └── uevent
```

Sysfs can be used as an additional way to interact with userspace. Setting up some driver params, for example.
Another useful thing – configure **UDEV** variables to set up correct permissions to the character device.
This can be done by setting uevent callback to sysfs class.

```
static int mychardev_uevent(struct device *dev, struct kobj_uevent_env *env)
{
    add_uevent_var(env, "DEVMODE=%#o", 0666);
    return 0;
}
```

```
...

mychardev_class = class_create(THIS_MODULE, "mychardev");
mychardev_class->dev_uevent = mychardev_uevent;
```

Now we got “*rw-rw-rw-*” permissions on each mychardev.

```
$ ls -l /dev/mychardev-*
crw-rw-rw- 1 root root 246, 0 Mar 14 12:24 /dev/mychardev-0
crw-rw-rw- 1 root root 246, 1 Mar 14 12:24 /dev/mychardev-1
```

Every user can read and write.

When a character device is no longer required it must be properly destroyed.

```
void mychardev_destroy(void)
{
    int i;

    for (i = 0; i < MAX_DEV; i++) {
        device_destroy(mychardev_class, MKDEV(dev_major, i));
    }

    class_unregister(mychardev_class);
    class_destroy(mychardev_class);

    unregister_chrdev_region(MKDEV(dev_major, 0), MINORMASK);
}
```

Device I/O functions

To interact with your device file, we need to set a few functions to the **struct file_operations.**

```
static int mychardev_open(struct inode *inode, struct file *file)
{
    printk("MYCHARDEV: Device open\n");
    return 0;
}

static int mychardev_release(struct inode *inode, struct file *file)
{
    printk("MYCHARDEV: Device close\n");
    return 0;
}

static long mychardev_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    printk("MYCHARDEV: Device ioctl\n");
    return 0;
}

static ssize_t mychardev_read(struct file *file, char __user *buf, size_t count, loff_t *offset)
{
    printk("MYCHARDEV: Device read\n");
    return 0;
}

static ssize_t mychardev_write(struct file *file, const char __user *buf, size_t count, loff_t *offset)
{
    printk("MYCHARDEV: Device write\n");
    return 0;
}
```

Now we can handle I/O requests.
If build and load the kernel module with this code and then run "*cat /dev/mychardev-0*" these messages will be printed in dmesg:

```
$ cat /dev/mychardev-0

$ sudo tail -n3 /var/log/messages
Mar 14 12:52:46 oleg-lab kernel: [244801.849652] MYCHARDEV: Device open
Mar 14 12:52:46 oleg-lab kernel: [244801.849665] MYCHARDEV: Device read
Mar 14 12:52:46 oleg-lab kernel: [244801.849672] MYCHARDEV: Device close
```

It's working.

To transfer some real data within `read/write` requests, we need to use special kernel functionality.
It's very dangerous or even impossible to do simple memory copying using *buf pointers.
Safe way is to use **copy_to_user()** and **copy_from_user()**

```
#include <linux/uaccess.h>

unsigned long copy_to_user(void __user *to, const void *from, unsigned long n);
unsigned long copy_from_user(void *to, const void __user *from, unsigned long n);
```

These functions perform additional checks of the permissions and memory regions before actual data access.

Let's modify our **mychardev_read().**

```
static ssize_t mychardev_read(struct file *file, char __user *buf, size_t count, loff_t *offset)
{
    uint8_t *data = "Hello from the kernel world!\n";
    size_t datalen = strlen(data);

    if (count > datalen) {
        count = datalen;
    }

    if (copy_to_user(buf, data, count)) {
        return -EFAULT;
    }

    return count;
}
```

It's always important to check how many bytes users want to read. If this size exceeds the prepared data's actual size – the user can read the kernel stack what can be a hole in the system security.

Now let's try to read 29 bytes from our character device.

```
$ head -c29 /dev/mychardev-1
Hello from the kernel world!
```

Of course, we can send to the user space not only strings but any other raw data structures.

Now **mychardev_write().**

```
static ssize_t mychardev_write(struct file *file, const char __user *buf, size_t count, loff_t *offset)
{
    size_t maxdatalen = 30, ncopied;
    uint8_t databuf[maxdatalen];

    if (count < maxdatalen) {
        maxdatalen = count;
    }

    ncopied = copy_from_user(databuf, buf, maxdatalen);

    if (ncopied == 0) {
        printk("Copied %zd bytes from the user\n", maxdatalen);
```

```
    } else {
        printk("Could't copy %zd bytes from the user\n", ncopied);
    }

    databuf[maxdatalen] = 0;

    printk("Data from the user: %s\n", databuf);

    return count;
}
```

It's also very important to verify how many bytes sending users and how many bytes we can accept.

Function copy_from_user returns the number of bytes that could not be copied. On success, this will be zero.
If some data could not be copied, this function will pad the copied data to the requested size using zero bytes.

Test:

```
$ echo "Hello from the user" > /dev/mychardev-1

$ sudo tail -n5 /var/log/messages
Mar 14 15:57:14 oleg-lab kernel: [255870.547447] MYCHARDEV: Device open
Mar 14 15:57:14 oleg-lab kernel: [255870.547466] Copied 20 bytes from the user
Mar 14 15:57:14 oleg-lab kernel: [255870.547468] Data from the user: Hello from the user
Mar 14 15:57:14 oleg-lab kernel: [255870.547468]
Mar 14 15:57:14 oleg-lab kernel: [255870.547472] MYCHARDEV: Device close
```

You may ask how to identify which device (mychardev-0 or mychardev-1) is used in a specific I/O process?
Since our Minor numbers are the same as device names we can get `Minor number` from the file inode using **struct file**.

```
MINOR(file->f_path.dentry->d_inode->i_rdev)
```

Let's print this value in the **read** and **write** functions and see what happens.

```
...
printk("Reading device: %d\n", MINOR(file->f_path.dentry->d_inode->i_rdev));

...
printk("Writing device: %d\n", MINOR(file->f_path.dentry->d_inode->i_rdev));
```

Result:

```
$ echo "Hello from the user" > /dev/mychardev-0

dmesg
Mar 14 16:02:08 oleg-lab kernel: [256164.495609] Writing device: 0

$ echo "Hello from the user" > /dev/mychardev-1

dmesg
Mar 14 16:02:08 oleg-lab kernel: [256164.495609] Writing device: 1
```

Few notes about ioctl.

```
static long mychardev_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
```

This utility function is used to pass some CMD as the number and some optional data as ARG.

You need to define some magic numbers used as CMD (and probably as ARG) somewhere in a separate header file, shared between driver code and user application code.

All implementation of the ioctl function is a simple switch case routine where you do something depending on the sent CMD.

Now **a complete example** of the Linux kernel module, which implements everything that we were discussed here.

```
#include <linux/init.h>
#include <linux/module.h>
```

```
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/kernel.h>
#include <linux/uaccess.h>
#include <linux/fs.h>

#define MAX_DEV 2

static int mychardev_open(struct inode *inode, struct file *file);
static int mychardev_release(struct inode *inode, struct file *file);
static long mychardev_ioctl(struct file *file, unsigned int cmd, unsigned long arg);
static ssize_t mychardev_read(struct file *file, char __user *buf, size_t count, loff_t *offset);
static ssize_t mychardev_write(struct file *file, const char __user *buf, size_t count, loff_t *offset);

static const struct file_operations mychardev_fops = {
    .owner      = THIS_MODULE,
    .open       = mychardev_open,
    .release    = mychardev_release,
    .unlocked_ioctl = mychardev_ioctl,
    .read       = mychardev_read,
    .write      = mychardev_write
};

struct mychar_device_data {
    struct cdev cdev;
};

static int dev_major = 0;
static struct class *mychardev_class = NULL;
static struct mychar_device_data mychardev_data[MAX_DEV];

static int mychardev_uevent(struct device *dev, struct kobj_uevent_env *env)
{
    add_uevent_var(env, "DEVMODE=%#o", 0666);
    return 0;
}

static int __init mychardev_init(void)
{
    int err, i;
    dev_t dev;

    err = alloc_chrdev_region(&dev, 0, MAX_DEV, "mychardev");

    dev_major = MAJOR(dev);

    mychardev_class = class_create(THIS_MODULE, "mychardev");
    mychardev_class->dev_uevent = mychardev_uevent;

    for (i = 0; i < MAX_DEV; i++) {
        cdev_init(&mychardev_data[i].cdev, &mychardev_fops);
        mychardev_data[i].cdev.owner = THIS_MODULE;

        cdev_add(&mychardev_data[i].cdev, MKDEV(dev_major, i), 1);

        device_create(mychardev_class, NULL, MKDEV(dev_major, i), NULL, "mychardev-%d", i);
    }
```

```
    return 0;
}

static void __exit mychardev_exit(void)
{
    int i;

    for (i = 0; i < MAX_DEV; i++) {
        device_destroy(mychardev_class, MKDEV(dev_major, i));
    }

    class_unregister(mychardev_class);
    class_destroy(mychardev_class);

    unregister_chrdev_region(MKDEV(dev_major, 0), MINORMASK);
}

static int mychardev_open(struct inode *inode, struct file *file)
{
    printk("MYCHARDEV: Device open\n");
    return 0;
}

static int mychardev_release(struct inode *inode, struct file *file)
{
    printk("MYCHARDEV: Device close\n");
    return 0;
}

static long mychardev_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    printk("MYCHARDEV: Device ioctl\n");
    return 0;
}

static ssize_t mychardev_read(struct file *file, char __user *buf, size_t count, loff_t *offset)
{
    uint8_t *data = "Hello from the kernel world!\n";
    size_t datalen = strlen(data);

    printk("Reading device: %d\n", MINOR(file->f_path.dentry->d_inode->i_rdev));

    if (count > datalen) {
        count = datalen;
    }

    if (copy_to_user(buf, data, count)) {
        return -EFAULT;
    }

    return count;
}

static ssize_t mychardev_write(struct file *file, const char __user *buf, size_t count, loff_t *offset)
{
    size_t maxdatalen = 30, ncopied;
    uint8_t databuf[maxdatalen];
```

```
    printk("Writing device: %d\n", MINOR(file->f_path.dentry->d_inode->i_rdev));

    if (count < maxdatalen) {
        maxdatalen = count;
    }

    ncopied = copy_from_user(databuf, buf, maxdatalen);

    if (ncopied == 0) {
        printk("Copied %zd bytes from the user\n", maxdatalen);
    } else {
        printk("Could't copy %zd bytes from the user\n", ncopied);
    }

    databuf[maxdatalen] = 0;

    printk("Data from the user: %s\n", databuf);

    return count;
}

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Oleg Kutkov <elenbert@gmail.com>");

module_init(mychardev_init);
module_exit(mychardev_exit);
```

And **Makefile** to build this code.

```
BINARY     := mychardev
KERNEL     := /lib/modules/$(shell uname -r)/build
ARCH       := x86
C_FLAGS    := -Wall
KMOD_DIR   := $(shell pwd)
TARGET_PATH := /lib/modules/$(shell uname -r)/kernel/drivers/char

OBJECTS := main.o

ccflags-y += $(C_FLAGS)

obj-m += $(BINARY).o

$(BINARY)-y := $(OBJECTS)

$(BINARY).ko:
    make -C $(KERNEL) M=$(KMOD_DIR) modules

install:
    cp $(BINARY).ko $(TARGET_PATH)
    depmod -a

uninstall:
    rm $(TARGET_PATH)/$(BINARY).ko
    depmod -a

clean:
    make -C $(KERNEL) M=$(KMOD_DIR) clean
```

Module building and loading

make && sudo insmod mychardev.ko

You should find two new devices: /dev/mychardev-0 and /dev/mychardev-1, and repeat all experiments from this article.

I hope this material will be helpful.
This code can be used as a basic pattern in some more complex driver project.

Thanks for reading!

Tagged character, chardev, driver, kernel, linux